CENTRO UNIVERSITÁRIO UNIFECAF
GRADUAÇÃO EAD

Gilberto Vagner Ferreira Cavalcante

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ÁGUAS LINDAS – GO.
2026
GILBERTO VAGNER FERREIRA CAVALCANTE

Data Structure Strategy and Implementation

O trabalho apresentado é sobre a avaliação da disciplina Data Structure Strategy and Implementation do Centro Universitário UniFECAF.

Tutor: **Vitor Jansen**

ÁGUAS LINDAS - GO

2026

SUMÁRIO

Sumário

PROJETO 1
QUESTÕES DO TEMA EXPLORADO 2
EXTRUTURA DO CÓDIGO 4
CONCLUSÃO 5

PROJETO

O trabalho tem base em analisar os desafios enfrentados pela "MegaStore" por estar com um problema de busca atual no seu sistema, que apresenta falhas de performance e problemas de precisão ao lidar com um catálogo de muito grande de produtos. A seguir, apresentam-se as respostas fundamentadas para as questões norteadoras do projeto.

QUESTÕES DO TEMA EXPLORADO

1. Quais são as principais causas da lentidão e imprecisão do sistema de busca atual da "MegaStore"?

O problema de lentidão pode ocorrer devido ao uso de métodos de busca tradicionais (como busca em sequência) que é complexo e custoso, tornando ineficiente quando a quantidade de dados atinge milhões de itens.

2. Quem são os principais afetados pela baixa performance do sistema de busca e como eles são afetados?

Os mais afetados são os clientes, que sentem o problema na hora de buscar um produto e abandonam o site ao não encontrar o que desejam fazendo a empresa sofrer com a perda de vendas, causando prejuízos.

3. Quais são as possíveis soluções para otimizar o sistema de busca da "MegaStore" e quais são os prós e contras de cada uma?

Tabelas Hash: Oferecem busca em tempo constante, sendo extremamente rápidas para acesso direto. Só que consome mais memória RAM para manter os índices.

Árvores de Busca: Eficientes para buscas por intervalo e organização por ordem. Busca ligeiramente mais lenta que a Hash, e mais complexo.

Grafos: Excelentes para sistemas de recomendação baseados em relacionamento entre produtos. Mas sua implementação é mais complexa e com maior custo computacional para grandes volumes.

4. Quais são as barreiras ou desafios para implementar cada uma das soluções propostas?

A barreira é garantir a necessidade da escalabilidade do sistema enquanto o catálogo cresce, o gerenciamento da memória para grandes volumes de dados fora o nível de aprendizado da linguagem Rust que é maior se comparado a Python, que por padrão exige maior cuidado no gerenciamento de memória (Ownership e Borrowing).

5. Como a utilização de tabelas hash em Rust pode contribuir para a otimização do sistema de busca da "MegaStore"?

Após consultas foi constatado que as tabelas hash permitem indexar o catálogo de um produto por uma chave (como ID ou nome). Em Rust, é altamente segura e eficiente, garantindo que o sistema suporte milhões de itens sem queda de desempenho.

6. Quais são os requisitos de hardware e software necessários para implementar a solução escolhida?

Como visto nas aulas, são necessárias as ferramentas do Rust (compilador rustc e gerenciador cargo). Para o hardware, é necessário um servidor com muita memória e processador com vários núcleos para processamento paralelo.

7. Como a solução proposta se integra com o restante da plataforma de e-commerce da "MegaStore"?

A resolução do problema deve ser implementada como um módulo e microserviço de busca que se comunica com o banco de dados principal para indexação.

8. Quais são os indicadores de desempenho (métricas) que podem ser utilizados para avaliar a eficácia da solução implementada?

Pelo que vi pode ser utilizados o tempo de resposta (latência da busca), a taxa de conversão de vendas (clientes que compram após a busca) e o uso de recursos computacionais (CPU e RAM) quando usado sob muita carga.

9. Quais são as implicações a longo prazo para a "MegaStore" caso o problema do sistema de busca não seja resolvido?

Caso não implemente as devidas correções a empresa vai enfrentar perdas de vendas e competitividade no mercado, sistema tecnológico defasado e queda de clientes.

EXTRUTURA DO CÓDIGO

Estrutura de Diretórios do Projeto

- trabalho-megastore/ (Pasta Raiz Principal)
    - Cargo.toml: Arquivo de configuração que contém as dependências e informações sobre o projeto em Rust.
    - README.md: Arquivo de documentação com as instruções de uso, exemplos e arquitetura do sistema.
    - Documentacao_MegaStore.pdf: O arquivo PDF contendo toda a documentação desenvolvida para o trabalho.
    - src/: Pasta que deve conter todo o código-fonte do sistema de busca, organizada em arquivos Rust.
        - main.rs: O arquivo principal onde fica a lógica de busca e o catálogo.

CONCLUSÃO

A implementação de um sistema de busca baseado em tabelas hash utilizando a linguagem Rust apresenta-se como a solução mais viável para a MegaStore, unindo a alta performance necessária para milhões de dados com a segurança e confiabilidade exigidas pela empresa.